

Real-Time Collaborative Document Editor: A Strategic Implementation Roadmap

1. Problem Comprehension and Strategic Scope

The challenge at hand requires engineering a real-time collaborative document editor analogous to Google Docs. In distributed systems terminology, this involves building a platform where multiple concurrent users can mutate a shared text state over an unreliable network without encountering data loss, race conditions, or permanent divergence.

The evaluation criteria set by the hackathon mentors emphasize the underlying distributed systems architecture rather than commercial polish. Mentors will strictly evaluate state consistency during simulated network partitions, the latency of cursor synchronization, the memory efficiency of the chosen data structures, and the reliability of offline synchronization. The objective is to build a functional Minimum Viable Product (MVP) that convincingly demonstrates these complex operations within a constrained eight-hour hackathon window, assuming a six-hour active coding period.

To succeed, first-year engineering teams must ruthlessly prioritize features. The development effort must be channeled exclusively toward satisfying the core architectural requirements while aggressively discarding tangential functionalities.

Feature Prioritization Matrix

The following table delineates the project scope, separating critical path requirements from tasks that introduce unnecessary technical risk.

Feature Category	Specific Functionality	Justification
Must-Have (Core MVP)	Conflict-Free Replicated Data Type (CRDT)	Mandatory for resolving concurrent edits without locking mechanisms or server-side transformation logic.
Must-Have (Core MVP)	WebSocket Synchronization	Required for low-latency state propagation and real-time cursor tracking.
Must-Have (Core MVP)	Local Browser Persistence	Satisfies the offline recovery

		requirement, allowing the application state to survive page reloads and network drops.
Must-Have (Core MVP)	Live Presence Indicators	Demonstrates real-time capability via colored cursors and active selection highlighting.
Nice-to-Have	Diagnostic Memory Dashboard	Exposing the CRDT memory footprint directly addresses the mentor evaluation guidelines and adds significant technical credibility.
Nice-to-Have	Basic Rich Text Formatting	Adding bold, italic, and list formatting proves the CRDT can handle nested document schemas, not just plain strings.
Strictly Avoid	User Authentication (OAuth/JWT)	Introduces massive scope creep. Teams should rely on a simple prompt asking for a display name.
Strictly Avoid	Database Deployment (PostgreSQL)	Provisioning cloud databases consumes valuable time. The system should rely exclusively on in-browser databases and in-memory server state for the prototype.
Strictly Avoid	Multiple Document Management	Building routing, folders, and access control is irrelevant to the core distributed systems

		challenge. The application should hardcode a single shared "room".
--	--	--

2. Technology Stack and Architectural Selection

Constructing a collaborative engine from first principles is a multi-year endeavor. A fundamental rule of hackathon engineering is to leverage established open-source libraries that abstract the most mathematically complex components. The recommended stack utilizes industry-standard tools that provide maximum reliability with minimum configuration.

Core Data Structure: Yjs

The prompt explicitly forbids Last-Write-Wins (LWW) conflict resolution, mandating either a CRDT or Operational Transformation (OT) approach. Operational Transformation requires a central server to dictate the absolute order of operations, which becomes geometrically complex to calculate when users disconnect and reconnect with divergent histories¹. CRDTs, conversely, treat every operation as a mathematically commutative action, allowing peers to merge updates independently³.

Among CRDTs, Yjs is the optimal choice for a hackathon. Yjs utilizes the Yet Another Transformation Approach (YATA) algorithm, which represents documents as doubly-linked lists and ensures that concurrent insertions never cross over each other, guaranteeing strong eventual consistency⁴.

CRDT Library	Performance Profile	Ecosystem Integration	Verdict
Yjs	Highly optimized memory footprint; fast loading times ⁶ .	Extensive out-of-the-box bindings for popular React editors ⁷ .	Recommended. Safest and most documented path for beginners.
Automerge	Retains a complete Git-like DAG of history, causing larger memory overhead ⁶ .	Powerful for complex JSON states, but bindings for rich text are less mature.	Discard. The memory overhead conflicts with the mentor evaluation criteria.
Loro	Exceptional benchmark	Extremely new ecosystem; requires	Discard. Introduces too much technical

	performance.	manual integration for many editor frameworks.	risk for an 8-hour timeline.
--	--------------	--	------------------------------

Frontend Editor: TipTap and React

Building a text editor interface that calculates exact cursor positions across different browser rendering engines is notoriously difficult. TipTap is a headless wrapper around the ProseMirror framework⁹. It is highly recommended because it separates the visual styling from the document schema and, crucially, maintains official first-party extensions for Yjs collaboration (@tiptap/extension-collaboration) and live cursors (@tiptap/extension-collaboration-cursor)¹⁰. The user interface will be wrapped in React, scaffolded via Vite for instantaneous hot-module reloading during development.

Networking: y-websocket (Node.js)

To synchronize the Yjs document state, clients must transmit their operations over a network. While WebRTC allows direct peer-to-peer connections, it frequently fails on enterprise or university Wi-Fi networks due to strict Network Address Translation (NAT) firewalls. WebSockets provide a persistent, full-duplex TCP connection to a central relay server¹². Using the y-websocket library allows the team to deploy a "dumb" Node.js server. The server does not need to understand the document; it merely receives binary CRDT updates from one client and broadcasts them to all others¹³.

Persistence: y-indexeddb

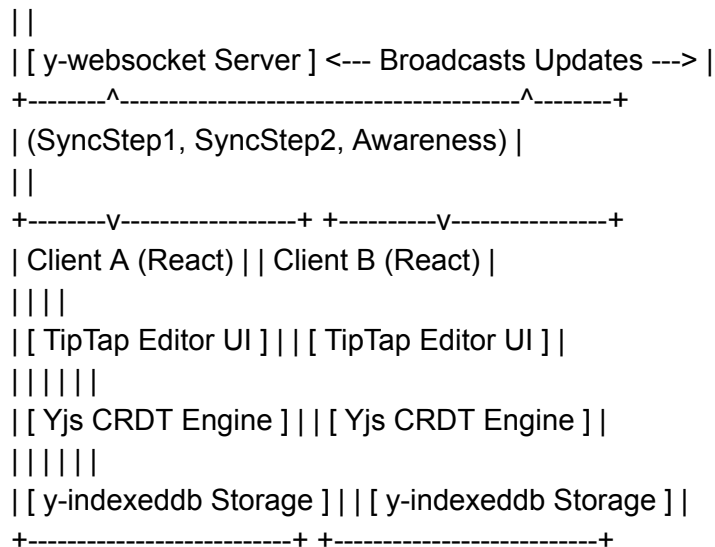
The evaluation guidelines demand offline sync performance and recovery after network partitioning. By utilizing y-indexeddb, the application will seamlessly stream all Yjs binary updates directly into the browser's native IndexedDB storage¹⁵. IndexedDB offers hundreds of megabytes of asynchronous storage capacity, far exceeding LocalStorage quotas¹⁷. If the browser tab is closed or the network drops, the exact document state is instantly rehydrated from the local disk upon the next launch, completely fulfilling the offline recovery requirement without the need to configure a cloud database.

3. System Architecture

The architecture functions as a state-synchronization loop. The underlying principle is that the "document" does not exist as a static string of text. Instead, it exists as a chronological sequence of individual insert and delete operations (represented as binary data) that are mathematically assembled into text on the user's screen¹⁹.

Data Flow Diagram

+-----+
| Node.js Relay Server |



Component Communication Mechanics

When an engineer interacts with the system, a highly choreographed sequence of events occurs. When the application loads, the local Yjs engine immediately queries y-indexeddb to load any previously saved offline work. Simultaneously, the application initiates a WebSocket handshake with the Node.js server.

This handshake relies on the Yjs synchronization protocol, which operates in two steps. In SyncStep1, the client sends a "State Vector"—a highly compressed mathematical summary indicating exactly which document updates it already possesses²⁰. The server forwards this to other clients, who analyze the State Vector, determine what information the new client is missing, and respond with SyncStep2, which contains only the precise binary updates required to bridge the gap²¹.

Once synchronized, any keystroke registered in the TipTap UI is converted by the Yjs engine into a micro-update. This update is saved to the local IndexedDB and broadcasted over the WebSocket¹⁴. Because Yjs uses the YATA algorithm, remote clients receive this update and calculate its exact intended position in the document without requiring the server to lock the file or dictate the order of operations². Live cursors are handled entirely separately via the "Awareness" protocol, which broadcasts ephemeral data (like mouse coordinates and user colors) that is deliberately excluded from the permanent document history to preserve memory²⁰.

4. Modular Project Breakdown

To ensure rapid execution, the repository must be strictly partitioned into four logical modules. This isolation prevents integration conflicts and allows the team to develop in parallel.

Module Designation	Core Responsibility	Required Technologies	Expected Outcome
--------------------	---------------------	-----------------------	------------------

1. Persistence & Engine	Manage the local CRDT state and offline storage.	yjs, y-indexeddb	A headless JavaScript object that retains text data across browser refreshes.
2. Interface & Editing	Provide the visual text surface and formatting tools.	React, tiptap/react, tiptap/starter-kit	A centered, styled text box where users can type, bold, and italicize text.
3. Communication Relay	Broadcast binary states and awareness data across the network.	Node.js, ws, y-websocket	A terminal process logging active WebSocket connections, capable of running locally or on a cloud host.
4. Synchronization Glue	Bind the UI to the CRDT engine and the network provider.	tiptap/extension-collaboration, tiptap/extension-collaboration-cursor	Multiple browser windows updating simultaneously with visible, colored user carets.

5. Detailed Eight-Hour Execution Timeline

The following timeline allocates six hours for intense, parallelized coding, leaving adequate margins for debugging, integration, and presentation refinement in the remaining two hours.

Phase 1: Foundation (Hours 0–1.5)

The objective of the first hour and a half is to scaffold the application and prove that the individual technologies function in isolation.

- **Hour 0–0.5 (Environment Setup):** The team initializes a monorepo containing a Vite-React client and a Node.js server. All dependencies from the tech stack are installed. The team verifies that the frontend development server and the backend Node.js script launch without fatal errors.
- **Hour 0.5–1.5 (UI & Relay Establishment):** The frontend engineer mounts a basic, standalone TipTap editor without collaboration features, applying CSS to visually simulate

a document. Concurrently, the backend engineer implements the y-websocket server logic and confirms via terminal logs that it can accept incoming traffic.

- *Fallback Strategy:* If the Node.js server fails to deploy to a cloud provider, the team must immediately pivot to running the server locally on a specific IP address for the remainder of the hackathon.

Phase 2: Core Synchronization (Hours 1.5–4)

This phase represents the primary technical hurdle. The team must successfully bridge the frontend editor to the backend relay using the CRDT engine.

- **Hour 1.5–3 (Yjs Integration):** The integration engineer instantiates the Y.Doc on the client and configures the WebsocketProvider to connect to the backend. The TipTap collaboration extensions are injected into the editor configuration, binding the UI to the Y.Doc.
- **Hour 3–4 (Presence Implementation):** The team configures the Awareness protocol. A randomized username and hex color are generated on load and fed into the collaboration-cursor extension.
- *Testing Gate:* By the end of Hour 4, two browser windows opened side-by-side must exhibit sub-50-millisecond synchronization of both text and colored cursors. If this fails, all other tasks must be halted until sync is achieved.

Phase 3: Resilience and Metrics (Hours 4–6)

With synchronization achieved, the system must be fortified to satisfy the specific evaluation criteria regarding network partitioning and memory constraints.

- **Hour 4–5 (Offline Persistence):** The team integrates y-indexeddb. The system is tested by severing the WebSocket connection, typing paragraphs of text, and restoring the connection to observe the automatic reconciliation of state.
- **Hour 5–5.5 (Diagnostic Tooling):** To directly address the judges' requirement regarding "memory footprint over long lifetimes," the team implements a visual byte-counter in the UI that displays the dynamically updating size of the encoded Yjs state vector.
- **Hour 5.5–6 (Refinement):** General UI cleanup. Margins are adjusted, typography is smoothed, and the application is prepared for a visual demonstration.

Phase 4: Chaos Testing and Presentation (Hours 6–8)

The final two hours are strictly reserved for breaking the application intentionally and practicing the pitch.

- **Hour 6–7 (Adversarial Testing):** The team enacts the "Chaos Phase." Members attempt to type over each other rapidly. The server is abruptly terminated and restarted. Large blocks of text are pasted simultaneously to verify that the YATA algorithm prevents data corruption.
- **Hour 7–8 (Demo Preparation):** Code is frozen. The team writes a three-minute script, assigns speaking roles, and rehearses the precise choreography of the demonstration.

6. Step-by-Step Implementation Guide

This guide translates the architecture into an executable sequence of commands and code configurations tailored for novice engineers.

Step 1: Scaffolding the Monorepo

The project requires a clear separation between the client code and the server code.

1. Create the root directory: `mkdir real-time-editor && cd real-time-editor`
2. Initialize the React frontend using Vite: `npm create vite@latest client -- --template react`
3. Initialize the Node.js backend: `mkdir server && cd server && npm init -y`

Step 2: Installing Backend Dependencies

The backend relies exclusively on WebSocket protocols.

1. Navigate to the server directory: `cd server`
2. Install the required libraries: `npm install ws y-websocket`
3. The `y-websocket` package contains a pre-built connection handler (`setupWSConnection`) that manages the complexities of broadcasting the binary CRDT updates, eliminating the need to write custom relay logic²³.

Step 3: Installing Frontend Dependencies

The frontend requires the editor framework and the CRDT engine.

1. Navigate to the client directory: `cd ../client`
2. Install the core packages: `npm install yjs y-websocket y-indexeddb`
3. Install the TipTap ecosystem: `npm install @tiptap/react @tiptap/starter-kit @tiptap/extension-collaboration @tiptap/extension-collaboration-cursor`¹⁰.

Step 4: Configuring the Development Environment

Ensure the project can be launched easily. In the `client/package.json`, the standard `npm run dev` command provided by Vite will be utilized to serve the frontend on port 5173. In the `server/package.json`, add a script: `"start": "node index.js"`.

Step 5: Writing the Code

The team will populate the files according to the templates provided in Section 8. The backend server will be instantiated first, followed by the React UI components, and finally the CSS styles required to render the cursors.

7. Project Folder Structure

Maintaining strict organizational hygiene prevents import errors and cognitive overload during the hackathon. The recommended file tree is minimalist and direct.

`real-time-editor/`

```
|— server/
|   |— package.json
|   |— index.js # Contains the WebSocket relay logic.
|— client/
|   |— package.json
|   |— index.html # The HTML entry point.
|   |— src/
```

- | | | — main.jsx # React DOM rendering.
- | | | — Editor.jsx # The core component bridging TipTap, Yjs, and WebSockets.
- | | | — styles.css # Contains the visual layout and the specific cursor CSS.
- | | — vite.config.js # Vite build configuration.
- | — README.md # Documentation for the mentors reviewing the repository.
- server/index.js: The only backend file. It mounts an HTTP server, upgrades it to a WebSocket server, and delegates message handling to the Yjs utility.
- client/src/Editor.jsx: The central nervous system of the application. It initializes the Y.Doc, connects the network and database providers, and renders the text editing surface.
- client/src/styles.css: Critical for visual functionality. Without the specific CSS rules defined here, the remote cursors injected by Yjs will not be visible in the DOM.

8. Implementation Code Templates

The following code snippets are engineered for simplicity and reliability. They abstract away edge cases to provide a stable hackathon foundation.

The Relay Server (server/index.js)

Location: server/index.js **Function:** This script establishes a standard HTTP server and upgrades incoming connections to the WebSocket protocol. The `setupWSConnection` method natively handles the Yjs SyncStep sequence and awareness broadcasting. **Execution:** Run `npm start` from the server directory.

JavaScript

```
const WebSocket = require('ws');
const http = require('http');
const { setupWSConnection } = require('y-websocket/bin/utils');
```

```
// Instantiate a basic HTTP server to accept initial requests
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end("Yjs WebSocket Synchronization Server is Active");
});
```

```
// Bind the WebSocket server to the HTTP server instance
const wss = new WebSocket.Server({ server });
```

```
// Delegate all incoming WebSocket connections to the Yjs utility
wss.on('connection', (conn, req) => {
  setupWSConnection(conn, req);
});
```

```
const PORT = process.env.PORT || 1234;
server.listen(PORT, () => {
  console.log(`WebSocket Server listening on port ${PORT}`);
});
```

The Collaborative Interface (client/src/Editor.jsx)

Location: client/src/Editor.jsx **Function:** This React component orchestrates the distributed systems logic. It creates the Yjs CRDT engine (Y.Doc), attaches local persistence, establishes the WebSocket connection, and mounts the TipTap editor configured to read and write to the CRDT¹⁰.

JavaScript

```
import React, { useEffect, useState } from 'react';
import { useEditor, EditorContent } from '@tiptap/react';
import StarterKit from '@tiptap/starter-kit';
import Collaboration from '@tiptap/extension-collaboration';
import CollaborationCursor from '@tiptap/extension-collaboration-cursor';
import * as Y from 'yjs';
import { WebsocketProvider } from 'y-websocket';
import { IndexeddbPersistence } from 'y-indexeddb';

// 1. Instantiate the central CRDT data structure
const ydoc = new Y.Doc();

// 2. Generate randomized presence data for the user
const randomName = `Engineer ${Math.floor(Math.random() * 1000)}`;
const randomColor = '#' + Math.floor(Math.random()*16777215).toString(16);

export default function Editor() {
  const [syncState, setSyncState] = useState('Disconnected');
  const [byteSize, setByteSize] = useState(0);

  useEffect(() => {
    // 3. Establish the network connection to the relay server
    const provider = new WebsocketProvider('ws://localhost:1234', 'hackathon-document', ydoc);

    // 4. Bind the CRDT to the browser's local database for offline recovery
    const persistence = new IndexeddbPersistence('hackathon-document', ydoc);
```

```

// Track network status for the UI
provider.on('status', (event) => setSyncState(event.status));

// Track the memory footprint of the document dynamically
ydoc.on('update', () => {
  setByteSize(Y.encodeStateAsUpdate(ydoc).byteLength);
});

// Cleanup network connections when the component unmounts
return () => {
  provider.destroy();
};
}, []);

// 5. Configure the TipTap editor instance
const editor = useEditor({
  extensions: [
    // Disable default history; the CRDT algorithm manages all undo/redo logic
    StarterKit.configure({ history: false }),
    Collaboration.configure({ document: ydoc }),
    CollaborationCursor.configure({
      provider,
      user: { name: randomName, color: randomColor },
    }),
  ],
});

return (
  <div className="document-wrapper">
    <header className="metrics-dashboard">
      <span className={`status-indicator ${syncState}`}>Network: {syncState}</span>
      <span className="memory-metric">CRDT Payload: {byteSize} bytes</span>
    </header>
    <div className="editor-surface">
      <EditorContent editor={editor} />
    </div>
  </div>
);
}

```

The Visual Presentation (client/src/styles.css)

Location: client/src/styles.css **Function:** While standard CSS handles the layout, specific

classes must be targeted to render the awareness data. TipTap injects empty tags to represent remote users; these must be styled to appear as colored vertical lines with floating nameplates¹⁰.

CSS

```
body {  
  background-color: #f4f5f7;  
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial,  
  sans-serif;  
}
```

```
.document-wrapper {  
  max-width: 850px;  
  margin: 40px auto;  
}
```

```
.metrics-dashboard {  
  display: flex;  
  justify-content: space-between;  
  margin-bottom: 10px;  
  font-size: 13px;  
  font-weight: 600;  
  color: #555;  
}
```

```
.status-indicator.connected { color: #2ecc71; }  
.status-indicator.disconnected { color: #e74c3c; }
```

```
.editor-surface {  
  background: white;  
  min-height: 800px;  
  padding: 40px 60px;  
  box-shadow: 0 10px 30px rgba(0,0,0,0.1);  
  border-radius: 4px;  
  outline: none;  
}
```

```
.editor-surface:focus-within {  
  box-shadow: 0 10px 30px rgba(0,0,0,0.15);  
}
```

```

/* Mandatory styles to expose the remote CRDT cursors */
.collaboration-cursor__caret {
  border-left: 2px solid #0d0d0d;
  border-right: 2px solid #0d0d0d;
  margin-left: -2px;
  margin-right: -2px;
  pointer-events: none;
  position: relative;
  word-break: normal;
}

.collaboration-cursor__label {
  position: absolute;
  top: -1.6em;
  left: -2px;
  font-size: 11px;
  font-style: normal;
  background-color: #0d0d0d;
  color: white;
  padding: 3px 6px;
  border-radius: 4px;
  white-space: nowrap;
  user-select: none;
}

```

9. Effective AI Utilization Strategy

Artificial Intelligence coding assistants are powerful accelerators for boilerplate generation, but they pose a severe threat if applied to specialized distributed systems logic. AI models frequently hallucinate operational transformation logic when queried about collaborative editing, which violates the CRDT requirement and causes catastrophic synchronization failures.

High-Value (Safe) AI Prompts

Engineers should leverage AI to accelerate tangential tasks, allowing human cognition to remain focused on the core architecture.

- **UI/UX Generation:** "Generate a CSS file that styles a generic <div> to look exactly like a centered A4 piece of paper with a subtle drop shadow, mimicking a word processor."
- **Error Diagnosis:** "I am receiving an ERESOLVE unable to resolve dependency tree error in NPM while installing @tiptap/extension-collaboration alongside yjs. How do I resolve peer dependency conflicts in React?"²⁷
- **Concept Clarification:** "Explain the difference between IndexedDB and LocalStorage, specifically regarding browser storage eviction policies and quota limits."¹⁸

- **Testing Automation:** "Provide a JavaScript snippet to generate 1,000 random characters programmatically inside a text input field to simulate heavy typing load."

Prohibited AI Tasks

Engineers must strictly avoid asking AI to engineer the synchronization mechanics.

- **Do not prompt:** "Write a custom function to merge two conflicting strings in real-time." The AI will likely generate a naive text-diffing script or attempt an elementary OT algorithm. The Yjs Y.Doc utilizes the YATA algorithm internally to calculate the exact insertion points based on Lamport clocks and doubly-linked list structures; attempting to override this with AI-generated logic will destroy the application⁴.
- **Do not prompt:** "Optimize the binary state vector sync protocol over WebSockets." The y-websocket library natively handles the SyncStep1 and SyncStep2 packet exchanges. Altering this flow will break the client-server handshake²⁰.

10. Strategic Team Division

A team of four first-year engineering students must parallelize their efforts to maximize the six-hour coding block. The architecture permits strict separation of concerns, ensuring members do not block one another.

Team Member Role	Core Responsibilities	Parallelization Strategy
Frontend Engineer	Implement the TipTap interface, apply typography CSS, and ensure the "A4 paper" visual layout is polished.	Can build the UI entirely offline using dummy text before the CRDT engine is wired up.
Backend Engineer	Scaffold the Node.js project, implement the y-websocket relay, and configure environment variables.	Can test the server independently using a generic WebSocket client (like Postman) to verify connectivity.
Systems Integrator	Manage the complex React useEffect hooks, initialize the Y.Doc, and bind y-indexeddb to the state.	Focuses purely on data flow. Can utilize a bare, unstyled <textarea> to prove the synchronization logic works before merging with the Frontend Engineer's polished UI.

QA & Presentation Lead	Orchestrate network partitioning tests, design the memory footprint dashboard, and author the presentation script.	Constantly monitors the repository, merging branches, identifying regressions, and ensuring the project aligns with the mentor evaluation criteria.
-----------------------------------	--	---

11. Testing Protocol and Debugging Strategy

Distributed systems fail in non-obvious ways. A systematic testing checklist ensures the prototype meets the rigorous mentor evaluation guidelines.

The System Validation Checklist

- **Latency Evaluation:** Open two browser windows on separate machines connected to the same network. Keystrokes must propagate in less than 50 milliseconds.
- **Presence Validation:** Ensure that when User A selects a block of text, User B sees that block highlighted in User A's assigned color³⁰.
- **Concurrency Stress Test:** Two engineers must attempt to type over the exact same word simultaneously. The YATA algorithm guarantees the text will not corrupt, but the UI must be monitored for cursor jitter².
- **Partition Simulation:** Disconnect User A's internet. User A types a paragraph. User A reconnects. The text must instantly populate on User B's screen without overriding User B's intervening edits.

15-Minute Emergency Debugging Protocol

If synchronization abruptly ceases, the team must execute this triage sequence immediately:

1. **Terminal Verification:** Check the Node.js terminal. If the process has crashed, restart it.
2. **Network Inspection:** Open Chrome DevTools -> Network -> WS (WebSockets). Verify that binary frames are actively being exchanged. If the connection is refused, verify the IP address and port mapping.
3. **The "Split-Brain" Fix:** If the application is running but changes are not syncing, a duplicate instance of the yjs library may have been loaded, causing the application to track two separate timelines. Delete the node_modules folder and package-lock.json, then execute npm install³¹.
4. **Database Purge:** During heavy development, local data can become corrupted. Navigate to Chrome DevTools -> Application -> IndexedDB -> Delete Database. Refresh the browser to force a clean SyncStep1 handshake with the server.

12. Hackathon Minimum Viable Product (MVP) Strategy

A successful hackathon project degrades gracefully. The team must define three tiers of completion to ensure a functional demonstration is always available.

Project Phase	Architectural Status	Demonstration Capability
Emergency MVP	Frontend React app with TipTap and a local Yjs instance. No backend server.	Proves the text editor can format text and handle complex schemas. Fails the multi-user requirement, but salvages UI points.
Target MVP	Frontend app connected to a locally hosted Node.js server via localhost:1234. y-indexeddb is active.	Satisfies all core requirements. During the demo, mentors must view the synchronization on the team's local machine using two browser windows side-by-side.
Full Version	Node.js server deployed to a cloud host. Frontend deployed to Vercel/Netlify.	Highly impressive. Mentors can access the URL on their personal smartphones and collaboratively type with the team live on stage.

Strategic Note: If the team struggles with cloud deployment for more than 45 minutes during Phase 4, they must abandon the Full Version and revert to the Target MVP. A flawless local demonstration scores exponentially higher than a broken cloud deployment.

13. High-Impact Enhancements (Impressing the Mentors)

To secure a winning position, the prototype must explicitly address the nuanced technical guidelines provided by the mentors.

1. Live Memory Footprint Telemetry The prompt explicitly mentions evaluating the "memory footprint of the CRDT structure." CRDTs store extensive metadata (tombstones, Lamport clocks) which causes document sizes to grow significantly over time⁸. By integrating a UI element that displays `Y.encodeStateAsUpdate(ydoc).byteLength`, the team proves they comprehend the theoretical drawbacks of CRDTs and are actively monitoring the performance overhead³³.

2. Visual Network Partition Simulation Rather than forcing mentors to manually disable their Wi-Fi to test offline capabilities, the team should engineer a dedicated "Simulate Network

Failure" toggle in the UI. When toggled, the application programmatically calls `provider.disconnect()` on the WebSocket. The UI background can shift to a warning color, allowing the team to type "offline" visibly. Toggling it back calls `provider.connect()`, triggering a highly visual, instant data reconciliation¹³.

14. Final Demonstration Strategy

The final pitch is a high-pressure, three-to-five-minute window. The demonstration must be highly theatrical, proving the technical claims through immediate visual evidence rather than slides.

The Demonstration Script and Choreography

- **Lead Presenter:** "Good afternoon. We were tasked with engineering a collaborative engine capable of synchronizing state across unreliable networks. Traditional systems rely on Operational Transformation, which scales poorly when handling offline latency. We engineered our solution using a Conflict-Free Replicated Data Type, specifically leveraging the YATA algorithm."
- *(Action: Two engineers aggressively type on the same sentence simultaneously. The screen displays the seamless integration of characters).*
- **Lead Presenter:** "As demonstrated, the system provides sub-50-millisecond state propagation with live presence tracking. However, the true test of a distributed system is partition tolerance."
- *(Action: The presenter clicks the custom 'Simulate Network Failure' button. The UI flashes 'OFFLINE').*
- **Lead Presenter:** "We have severed the WebSocket connection. I am now typing locally. The CRDT engine is currently buffering these mutations directly to the browser's IndexedDB, ensuring zero data loss¹⁵."
- *(Action: The presenter clicks 'Reconnect').*
- **Lead Presenter:** "Upon reconnection, our system executes a state vector exchange. It calculates the exact mathematical delta required and merges my offline edits globally without locking the document or overwriting my colleague's intervening work²."
- **Lead Presenter:** "Finally, to address long-term stability, we implemented real-time telemetry monitoring the binary footprint of the document state, ensuring the sequence data remains highly compressed. Thank you."

Contingency Plan: If the WebSocket server crashes mid-presentation, the presenter must pivot immediately: "You are currently witnessing an unplanned network partition. Notice how the editor remains fully interactive. Our local persistence model ensures that no data is lost during this outage."

15. The Final Countdown Checklist

Strict adherence to a deployment checklist prevents catastrophic last-minute failures.

Time Remaining	Required Actions	Verification
Before Coding	Environment baseline established.	Node.js and NPM versions are synchronized across all team laptops.
After Target MVP	Code freezing.	All critical path logic is committed to the main branch. Experimental features are developed strictly on isolated branches.
1 Hour Before	Resilience verification.	node_modules is deleted and reinstalled via npm install to guarantee the package.json accurately reflects all dependencies.
10 Mins Before	Launch sequence.	Development servers are initiated. Two browser windows are staged side-by-side. The team ceases touching the keyboards until the mentors arrive.

This report is for informational and educational purposes regarding software architecture and distributed systems design. For decisions impacting production-level data integrity, security, or enterprise deployment, consult a qualified software architect or security professional.

Works cited

1. Collaborative Text Editing with Eg-walker: Better, Faster, Smaller, <https://arxiv.org/html/2409.14252v1>
2. System Design: Real-Time Collaborative Editor - Cracking Walnuts, <https://crackingwalnuts.com/post/collaborative-editor-system-design>
3. CRDTs: How Distributed Systems Agree Without Asking Permission, https://dev.to/dev_pedro/crdts-how-distributed-systems-agree-without-asking-permission-37gc
4. (PDF) Yjs: A Framework for Near Real-Time P2P Shared Editing on, https://www.researchgate.net/publication/290390486_Yjs_A_Framework_for_Near

[Real-Time P2P Shared Editing on Arbitrary Data Types](#)

5. Deep dive into Yrs architecture - Bartosz Sypytkowski, <https://www.bartoszsypytkowski.com/yrs-architecture/>
6. Yjs vs Automerge vs Loro: CRDT Libraries 2026 - PkgPulse, <https://www.pkgpulse.com/guides/yjs-vs-automerge-vs-loro-crdt-libraries-2026>
7. Yjs - Best of JS, <https://bestofjs.org/projects/yjs>
8. JS/WASM Benchmarks - Loro CRDT, <https://loro.dev/docs/performance>
9. @tiptap/extension-collaboration-cursor - NPM, <https://www.npmjs.com/package/@tiptap/extension-collaboration-cursor>
10. how-to-create-a-collaborative-text-editor-with-tiptap-yjs-nextjs-and, <https://github.com/liveblocks/liveblocks/blob/main/guides/pages/how-to-create-a-collaborative-text-editor-with-tiptap-yjs-nextjs-and-liveblocks.mdx?plain=1>
11. Tracked Changes with Collaboration example - Tiptap, <https://tiptap.dev/docs/tracked-changes/examples/collaboration>
12. Websocket Security Best Practices and Checklist - Invicti, <https://www.invicti.com/blog/web-security/websocket-security-best-practices>
13. yjs/y-websocket | DeepWiki, <https://deepwiki.com/yjs/y-websocket>
14. Building Real-time Collaborative Apps with Yjs & WebSockets, <https://khadervali.com/building-real-time-collaborative-apps-yjs-websockets/>
15. y-indexeddb - Yjs Docs, <https://docs.yjs.dev/ecosystem/database-provider/y-indexeddb>
16. Offline-First Collaboration: Persisting Yjs and Editor.js State with, <https://ashuchoudhury.in/blog/offline-first-collaboration-persisting-yjs-and-editor-js-state-with-indexeddb>
17. IndexedDB Database in React Apps - The Power of RxDB, <https://rxdb.info/articles/react-indexeddb.html>
18. IndexedDB API - MDN Web Docs - Mozilla, https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API
19. Yjs Fundamentals - Part 1: Theory | by Dovetail Engineering - Medium, <https://medium.com/dovetail-engineering/yjs-fundamentals-part-1-theory-232a450dad7b>
20. Yjs Fundamentals — Part 2: Sync & Awareness | Dovetail Engineering, <https://medium.com/dovetail-engineering/yjs-fundamentals-part-2-sync-awareness-73b8fab2233>
21. Optimizing Yjs first load at Chronicle - Anik Das, <https://anikd.com/blog/optimizing-yjs-first-load/>
22. Awareness | Tiptap Collaboration Docs, <https://tiptap.dev/docs/collaboration/core-concepts/awareness>
23. Deploying server with y-websocket - Yjs Community, <https://discuss.yjs.dev/t/deploying-server-with-y-websocket/1791>
24. Hocuspocus Provider Examples | Tiptap Collaboration Docs, <https://tiptap.dev/docs/hocuspocus/provider/examples>
25. Install Collaboration | Tiptap Collaboration Docs, <https://tiptap.dev/docs/collaboration/getting-started/install>
26. CollaborationCaret extension | Tiptap Editor Docs,

- <https://tiptap.dev/docs/editor/extensions/functionality/collaboration-caret>
27. [Bug]: Installing @tiptap/extension-collaboration-cursor causes,
<https://github.com/ueberdosis/tiptap/issues/4493>
 28. yjs/INTERNALS.md at main - GitHub,
<https://github.com/yjs/yjs/blob/main/INTERNALS.md>
 29. y-protocols/PROTOCOL.md at master · yjs/y-protocols - GitHub,
<https://github.com/yjs/y-protocols/blob/master/PROTOCOL.md>
 30. Selection extension | Tiptap Editor Docs,
<https://tiptap.dev/docs/editor/extensions/functionality/selection>
 31. Yjs best practices and tips | Liveblocks guide,
<https://liveblocks.io/docs/guides/yjs-best-practices-and-tips>
 32. Why you can't delete Yjs documents | Liveblocks guide,
<https://liveblocks.io/docs/guides/why-you-cant-delete-yjs-documents>
 33. Measuring the size of a YDoc - Yjs Community,
<https://discuss.yjs.dev/t/measuring-the-size-of-a-ydoc/181>
 34. Document Updates - Yjs Docs, <https://beta.yjs.dev/docs/api/document-updates/>